Primeiro contato com Hazelcast

Soma distribuída baseada em um exemplo oficial

Integrantes: preencher nomes e matrículas

Universidade Federal de Ouro Preto
BCC362 — Sistemas Distribuídos

Trabalho Prático 01

O que é Hazelcast?

Ideia central

Compartilhar dados e executar tarefas em um conjunto de processos.

- **IMap**: um mapa chave–valor distribuído.
- **IExecutorService**: envia tarefas para os membros.
- Usos: cache, sessões, contadores e estatísticas.
- Nosso ambiente: Java 17 e Hazelcast 5.6.0.

Fonte: Hazelcast: Java Executor Service e Making Your Map Data Safe.

Quando usar e quais são os limites?

Aplicações possíveis

- Dados compartilhados em memória.
- Acesso de baixa latência.
- Agregação e processamento em vários membros.

Custos e cuidados

- RAM e comunicação têm custo.
- Backup em RAM não é persistência.
- Um cluster não é necessário para uma soma pequena.

Nosso objetivo é aprender os conceitos com um algoritmo fácil de conferir.

Mercado e dimensão dos clusters

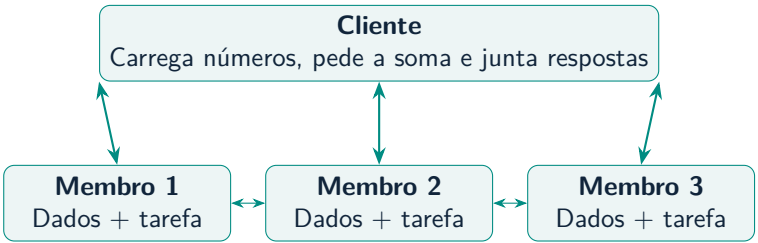
- **British Gas/Hive**: caso de IoT com **20 nós**.
- **BNP Paribas Bank Polska**: caso bancário; fornecedor relata 84 mil requisições/s.
- **Benchmark Jet 4.3**: 45 nós EC2, 720 vCPUs; 1 bilhão de eventos/s e p99 de 26 ms.

Qual é a maior aplicação?

Não encontramos evidência para afirmar um recorde mundial.
Benchmark não é aplicação comercial; nós não implicam máquinas físicas.

Fonte: Casos e benchmark do fornecedor, links no relatório. Números históricos, não medidos pelo grupo.

Cliente e membros: quem faz o quê?



- JVMs separadas se comunicam por TCP.
- Membros guardam partições; o cliente reúne os resultados.
- Nesta demonstração, todos os processos usam o mesmo host.

Partições e cópias de segurança

1. A chave é associada a uma partição por **hash**.
2. Uma tabela indica o proprietário de cada partição.
3. O mapa mantém **um backup síncrono** em outro membro.
4. Entrada ou saída de membros provoca redistribuição.

Neste projeto

271 partições padrão. Um membro sozinho não tem cópia em outro membro.

Fonte: Hazelcast: Data Partitioning e Making Your Map Data Safe.

Camadas e serviços

Camada	Responsabilidade
Aplicação	Carregar números, somar parciais e validar.
Serviços	IMap e IExecutorService.
Gestão do cluster	Descoberta, membros, partições e migração.
Infraestrutura	JVM, RAM, threads, serialização e TCP.

O cliente combina as respostas; os membros distribuem armazenamento e cálculo.

O que acontece quando o cluster muda?

Queda de um membro

Backup protege os dados.

Uma tarefa pode falhar.

A próxima rodada **refaz a soma inteira**.

Entrada de um membro

O novo membro recebe partições.

Rodadas seguintes usam o cluster atualizado.

O cliente continua aberto.

Pode haver pausa ou rodada descartada. Não prometemos latência zero, persistência ou execução exatamente uma vez.

A aplicação: somar de 1 até N

Exemplo pequeno

$1 + 2 + 3 + \cdots + 10 = 55$

1. Guardar os números no mapa `numeros`.
2. Cada membro soma suas chaves locais.
3. O cliente soma as parcelas e confere o total.

Origem: exemplo `SumTask` da documentação oficial.
Conceito: redução por soma; base para totais e estatísticas.
Validação: $N(N+1)/2$, sem substituir o cálculo distribuído.

A tarefa: poucas linhas para entender

```
IMap<Integer, Long> numeros =
    hazelcast.getMap("numeros");
long soma = 0;
for (Integer chave : numeros.localKeySet()) {
    soma += numeros.get(chave);
}
return soma;
```

- `localKeySet()`: chaves primárias deste membro.
- `Callable<Long>`: tarefa que retorna um número.
- `Serializable`: a tarefa pode ser enviada pelo Hazelcast.

Fonte: Trecho central adaptado de SumTask; código completo verifica valor indisponível.

O cliente: juntar as respostas

```
var respostas = hz.getExecutorService("soma")
    .submitToAllMembers(new SomaTask());
long total = 0;
for (var resposta : respostas.values()) {
    total += resposta.get();
}
```

- Uma tarefa por membro; `get()` espera a resposta.
- Parcelas diferentes, mesmo total: por exemplo $12 + 15 + 28 = 55$.
- Usamos `long`: a soma até 100.000 não cabe em `int`.

O que ocorre com entradas maiores?

Tempo de soma em milissegundos — menor é melhor

N	1 membro	2 membros	3 membros
1.000	94,9	49,5	36,8
10.000	677,4	295,9	295,9
100.000	5771,2	2907,5	2401,4

- Mediana de 3 medições após aquecimento; 27 medições ao todo.
- JVMs no mesmo host; carga separada no relatório.
- O custo do mapa e da coordenação limita os ganhos.
- Um membro usa Hazelcast: não é uma referência Java pura.

Demonstração: cliente aberto durante as mudanças

1. Mostrar `estado`: três membros, cópias seguras.
2. `somar 100000`: conferir **5.000.050.000**.
3. `acompanhar 100000`: encerrar o membro 5703.
4. Observar resultado válido com dois membros.
5. Iniciar membro 5704; observar resultado com três membros.

Evidência

Testes passaram com o mesmo processo cliente: falha, nova soma e expansão.

Perguntas e fontes

- Por que cada membro usa `localKeySet()`?
- O que o backup recupera e o que precisa ser recalculado?
- Por que adicionar membros não elimina todos os custos?

Fontes principais

Hazelcast: Java Executor Service — exemplo SumTask
Hazelcast: Making Your Map Data Safe
Hazelcast: Data Partitioning

Casos comerciais e demais referências: `docs/REFERENCIAS.md`.
Requisitos: BCC362_TP01.pdf, opção framework.